

StudyHubAi

<SHA> -

<StudyHubAi>

Documento de Visão

Índice

1	INTRODUÇÃO	5
1.1	Objetivo	5
1.2	Escopo	5
1.3	Fora do Escopo	5
1.4	Definições, Acrônimos, e Abreviações	5
1.5	Referências	5
2	CONTEXTO DO NEGÓCIO	6
2.1	Relato do Problema	6
2.2	Riscos Identificados	6
3	STAKEHOLDERS DO PROJETO	7
3.1	Papéis e Responsabilidades	7
3.2	Relação dos Stakeholders	7

4	CARACTERÍSTICAS DO PRODUTO	8
5	RESTRIÇÕES	9
5.1	Linguagens de Programação	9
5.2	Padrões de Desenvolvimento	9
5.3	Requisitos Não Funcionais	9
6	INTEGRAÇÃO GenAI.....	10
6.1	Estratégia de IA Generativa.....	10

1 INTRODUÇÃO

1.1 Objetivo

O objetivo deste documento é apresentar a visão geral do projeto StudyHubAI, uma plataforma web inteligente que visa transformar a forma como estudantes se conectam, colaboram e aprendem em conjunto. Este documento descreve o propósito, escopo, funcionalidades, stakeholders, riscos e restrições associadas ao sistema.

1.2 Escopo

O StudyHubAI é uma aplicação web desenvolvida com Flask e Python que permite a criação e gestão de grupos de estudo colaborativos. Inclui funcionalidades de autenticação, partilha de materiais, comunicação em tempo real, acompanhamento de progresso e um assistente de IA para apoio ao estudo, utilizando técnicas de processamento de linguagem natural.

1.3 Fora do Escopo

- Desenvolvimento de aplicação móvel nativa (iOS/Android) nesta versão (será Web Responsiva).
- Integração com sistemas de pagamento.
- Videochamadas em tempo real integradas na plataforma (será usado apenas chat de texto).
- Integração automática com o Moodle/LMS da universidade (planeado para a versão 2.0).

1.4 Definições, Acrônimos e Abreviações

- **LLM (Large Language Model):** Modelo de linguagem grande, tipo de inteligência artificial usado para processar e gerar texto (ex: Llama 3).
- **GenAI (Generative AI):** Inteligência Artificial Generativa.
- **MVP (Minimum Viable Product):** Produto Mínimo Viável, a versão mais simples do sistema que traz valor ao utilizador.
- **Kanban:** Método visual para gestão de fluxo de trabalho.
- **Pull Request (PR):** Pedido para fundir alterações de código no repositório principal.

1.5 Referências

Repositório GitHub: [Pedrocarv007/ESTG-ES2-Final-project: projeto final de engenharia de software](https://github.com/Pedrocarv007/ESTG-ES2-Final-project)

2 CONTEXTO DO NEGÓCIO

2.1 Relato do Problema

Os estudantes frequentemente enfrentam dificuldades em organizar grupos de estudo eficazes, compartilhar materiais e acompanhar o progresso individual e coletivo. Além disso, a falta de uma plataforma integrada que combine colaboração, gestão de conteúdo e apoio inteligente limita a produtividade e o engajamento acadêmico. O StudyHubAI resolve esses problemas oferecendo uma solução centralizada que promove colaboração ativa, partilha de recursos e suporte via assistente inteligente.

2.2 Riscos Identificados

A tabela a seguir apresenta os riscos identificados nessa etapa.

Descrição do Risco	Possível Solução	Probabilidade	Impacto
Latência na resposta da IA (Ollama a correr localmente pode ser lento em PCs mais fracos).	Implementar indicadores de carregamento ("spinners") e processamento assíncrono.	Alta	Médio
Dificuldade na integração do Git (Conflitos de código ao trabalhar em grupo).	Seguir rigorosamente o fluxo de branches e fazer Pull Requests pequenos.	Média	Alto
Escopo indefinido (Querer adicionar demasiadas funcionalidades).	Focar estritamente no MVP (User Stories prioritárias) definido no Kanban	Média	Alto
Curva de aprendizagem (Dificuldades com Flask/SQLAlchemy).	Programação em par (Pair Programming) e partilha de conhecimento.	Baixa	Médio

3 STAKEHOLDERS DO PROJETO

3.1 Papéis e Responsabilidades

Equipa de Desenvolvimento:

- Desenvolver o Frontend (HTML/CSS/JS) e Backend (Flask).
- Gerir a base de dados e migrações.
- Configurar e integrar o modelo de IA local.
- Realizar testes unitários e de aceitação.

Papel	Responsabilidades
Cliente	Patrocinar o projeto; Validar requisitos e entregas Garantir alinhamento com objetivos educacionais
Product Owner	Definir o escopo do sistema; Representar o cliente no desenvolvimento do projeto. Priorizar funcionalidades
Scrum Master	Facilitar cerimónias ágeis Remover impedimentos Assegurar o cumprimento do cronograma
Equipa de desenvolvimento	Implementar funcionalidades Criar e testar código Garantir qualidade e desempenho

3.2 Relação dos Stakeholders

Nome do Profissional	Papel
Toacy Oliveira	Scrum Master/Stakeholder
Pedro	Developer
Lara	Developer
Thaissa	Developer
Sarah	Developer

4 CARACTERÍSTICAS DO PRODUTO

Autenticação e Perfis

- **Funcionalidade 1:** Registo e login de utilizadores
- **Funcionalidade 2:** Perfis personalizados de estudantes
- **Funcionalidade 3:** Gestão de preferências de estudo

Gestão de Grupos

- **Funcionalidade 1:** Criação de grupos por disciplina/matéria
- **Funcionalidade 2:** Sistema de convites e aprovações
- **Funcionalidade 3:** Definição de objetivos e metas do grupo
- **Funcionalidade 4:** Calendário de sessões de estudo

Partilha de Recursos

- **Funcionalidade 1:** Upload e organização de materiais de estudo
- **Funcionalidade 2:** Categorização por tipo de conteúdo
- **Funcionalidade 3:** Sistema de comentários e avaliações
- **Funcionalidade 4:** Controlo de versões de documentos

Assistente IA Integrado

- **Funcionalidade 1:** Esclarecimento de dúvidas em tempo real
- **Funcionalidade 2:** Sugestões de estudo personalizadas
- **Funcionalidade 3:** Geração de resumos automáticos
- **Funcionalidade 4:** Recomendações de recursos complementares

Analytics e Progresso

- **Funcionalidade 1:** Métricas de participação individual
- **Funcionalidade 2:** Estatísticas de performance do grupo
- **Funcionalidade 3:** Relatórios de progresso de aprendizagem

5 RESTRIÇÕES

5.1 Linguagens de Programação

O projeto utiliza **Python 3.10+** como linguagem principal.

- **Backend:** Framework **Flask** para a lógica de servidor e API.
- **Base de Dados:** **SQLAlchemy** (ORM) com **SQLite** para desenvolvimento (fácil portabilidade).
- **Frontend:** **HTML5**, **CSS3**, **JavaScript** e **Bootstrap 5** para estilização responsiva.
- **IA:** Integração com **Ollama**.

5.2 Padrões de Desenvolvimento

Para garantir a qualidade e organização do código, a equipa segue os seguintes padrões:

- **Arquitetura MVT (Model-View-Template):** Padrão nativo do Flask para separar a lógica de dados, controlo e interface.
- **Git Flow Simplificado:** Uso de Feature Branches para cada nova funcionalidade, que são fundidas na branch main através de Pull Requests após revisão.
- **PEP 8:** Guia de estilo de código Python para manter a legibilidade.
- **Commits Semânticos:** Mensagens de commit claras (ex: feat: adicionar login, fix: corrigir erro no chat).

5.3 Requisitos Não Funcionais

- **Segurança:** As palavras-passe dos utilizadores devem ser armazenadas como hashes seguros (nunca em texto limpo).
- **Usabilidade:** A interface deve ser responsiva, adaptando-se a ecrãs de desktop e tablets.
- **Feedback do Sistema:** O sistema deve informar o utilizador sobre o estado das ações (ex: mensagens de sucesso, erro, ou "a carregar" durante a resposta da IA).
- **Privacidade:** Os dados sensíveis (materiais de estudo) processados pela IA não devem ser enviados para servidores externos públicos.

6 Integração GenAI

6.1 Estratégia de IA generativa

O módulo "Mimi AI" utiliza Grandes Modelos de Linguagem (LLMs) para apoiar o estudante.

- **Modelo:** Llama 3 (via Ollama).
- **Processamento:** Local, garantindo privacidade e funcionamento sem custos de API externa.
- **Integração:** O backend Flask atua como intermediário, recebendo o prompt do utilizador, enviando-o para o serviço Ollama local e devolvendo a resposta formatada para o frontend.